

UNIVERSITY OF GONDAR

COLLEGE OF INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

DBMS PROJECT

LIBRARY MANAGEMENT SYSTEM

NO	NAME	ID_NO	ROLE
1	MIKIAS MULU	GUR/02007/15	PROJECT MANAGER
2	HILINA ASRES	GUR/02222/15	DOCUMENTATION SPECIALIST
3	LATJOR SIMON	GUR/02798/15	SQL DEVELOPER
4	SAMUEL TEREFE	GUR/01871/15	DATABASE DESIGNER
5	BLEN TEGEGNEWORK	GUR/01924/15	DATA ANALYST

SUMMTTED TO:MR GETNET T.

SUMMITTED DATE: 20,2024

GONDAR, ETHIOPIA

Acknowledgment

We would like to express our deepest gratitude to our instructor, Mr. Getnet T, for their invaluable guidance, support, and encouragement throughout the development of this library management system project. Their expertise and insights have been instrumental in shaping our understanding and approach to this project.

We extend our sincere thanks to our classmates and peers for their constructive feedback and collaborative spirit, which have greatly contributed to the success of this project. Their willingness to share knowledge and ideas has been a source of inspiration and motivation.

We are also grateful to the library staff and patrons who provided us with practical insights and real-world scenarios, helping us to design a system that meets the needs of its users effectively. Their input has been crucial in ensuring the relevance and usability of our project.

Lastly, we would like to acknowledge the unwavering support of our families and friends, whose encouragement and understanding have been a constant source of strength throughout this journey.

Thank you all for your contributions and support.

Table of Contents

Acknowledgment 1

Abstract 1

Chapter one 2

Introduction of Project 2

1.1 Background 2

1.2 Statement of Problem 5

1.3 Objective 6

1.3.1 General objectives 6

1.3.2 Specific objective 6

1.4 Methodology 7

Chapter 2 10

Database Designing 10

2.1 Database Modeling 10

Core Components of Database Modeling 11

2.2 Business Rules and Requirements for the Library Management System 12

2.3 Building Block of the Database 14

2.4 ER Diagram 17

2.5 Relationships 18

2.5 Relational Model: Mapping ER Diagram to Relational diagram Model 20

2.6 Normalization 23

Chapter 3 34

Database Implementation 34

3.2 Whole SQL Command to develop LMS database with some inserted values 35

3.3 Relational diagram 39

Reference 44

Abstract

The project focuses on the development and implementation of a Library Management System (LMS), designed to modernize library operations through a digital platform. The system aims to streamline library processes such as book cataloging, user management, loan tracking, and overdue fine calculations. By employing relational database models and SQL-based solutions, the LMS enhances efficiency, accuracy, and user experience. The methodology includes comprehensive data collection, entity-relationship modeling, and normalization techniques. The resulting system ensures robust data integrity and operational scalability, meeting the evolving needs of modern libraries.

chapter one



Introduction of Project

Libraries play a critical role in education and research as hubs of knowledge. The increasing complexity of managing physical and digital resources poses challenges for traditional library systems. This chapter introduces the concept of a Library Management System (LMS) as a solution to these challenges. It explores the background of the project, the problem it addresses, and the objectives it aims to achieve.

1.1 Background

Libraries have long been essential repositories of knowledge, playing a crucial role in education and research. Traditionally, they have managed extensive collections of books, journals, and other resources manually. However, with the growing volume of information and user demand, libraries face significant challenges in maintaining and accessing book records efficiently.

Library Management System is an application which refers to library systems which are generally small or medium in size. It is used by librarian to manage the library using a computerized system where he/she can record various transactions like issue of books, return of books, addition of new books, addition of new patron etc.

Survey of Technologies

Through improved functionality, reliability, and user satisfaction, the integration of such technologies ensures that the Library Management System satisfies the needs modern libraries. The system can turn routine library tasks into a more efficient, automated procedure by utilizing the right tools and techniques. Several technologies that meet the demands for modern libraries must be integrated for a library management system to be successful. Strong database systems are essential for the effective management of library records. Modern Relational Database Management Systems (RDBMS) are among the popular database technologies. These systems define, modify, and query data using SQL (Structured Query Language) and structured data models. The typical programming language for administering and working with relational databases is called SQL (Structured Query Language). It offers strong commands for data creation, updating, and retrieval and high availability, essential for a dynamic environment like library management.

Title-based Area

This project belongs to the domain of **Library Management Systems**, aiming to revolutionize the way libraries operate by introducing a modern, centralized digital solution. It focuses on streamlining core functions such as cataloging books, managing member records, tracking borrowing and returns, monitoring overdue items, and automating fine collection. Additionally, it ensures seamless inventory oversight and efficient data handling. By replacing traditional manual processes with an intelligent system, this project aspires to enhance operational efficiency, minimize errors, and elevate the experience for both library patrons and staff.

Problem and Proposed Solution

In the modern world, libraries remain essential hubs of knowledge and learning. However, traditional methods of library management are increasingly inadequate in meeting the growing needs of users and administrators. Libraries often face significant challenges in organizing large collections, accurately tracking borrowed and returned items, managing overdue materials, and calculating fines. Manual systems, prone to errors and inefficiencies, can lead to misplaced books, delays in services, and user dissatisfaction. Additionally, the absence of centralized data makes it difficult for library staff to effectively monitor inventory or make data-driven decisions, further complicating operations.

To address these challenges, the proposed solution is an advanced **Library Management System (LMS)** designed to revolutionize library operations. This comprehensive digital platform integrates every aspect of library management into a unified system. By automating key processes such as cataloging, book lending, return tracking, fine calculations, and inventory monitoring, the LMS eliminates manual errors and improves operational efficiency.

The system also provides library administrators with tools for real-time tracking of inventory, user records, and analytical insights to optimize decision-making and resource allocation. For library users, the LMS offers a user-friendly interface with features such as online book searches, personalized user profiles, and automated reminders for due dates or overdue materials.

By streamlining processes and enhancing accessibility, the proposed Library Management System is designed to modernize library operations, minimize errors, and provide an improved experience for both users and administrators. This innovative approach ensures that libraries can continue to serve as vital resources in an increasingly digital age.

1.2 Statement of Problem

The primary challenge that libraries encounter is the cumbersome process of managing vast collections of resources. Manual record-keeping can lead to misplacement, inaccuracies, and inefficient retrieval of books, ultimately affecting the user experience. Additionally, tracking borrowed books and managing overdue fines can be labor-intensive and prone to errors.

The problem occurred before having computerized system includes :

- Because of the human environment, files are usually lost when digital systems are not used. Records may occasionally be lost as a result of human error.
- Damaged files are always lost due to an accident, such as a member unintentionally dumping water on the file, when a computerized

system is not available. In addition, natural disasters like fires or floods could harm the files. Division of Computer Science and Engineering Library Management System Record

- Searching is always challenging in the absence of a computerized system, especially when there are a lot of records.
- Space-consuming: If no electronic system is put in place, the amount of space needed for the physical storage of files and records grows as the number of records grows.
- Costly because there is no automated system, each record paper must be added, raising the expense of library management.

1.3 Objective

1.3.1 General objectives

The general objective of this project is to develop a Library Management System (LMS) that streamlines the management of book records and enhances access to library resources.

1.3.2 Specific objective

The system aims to:

- **Enhancement of performance and control**

  the system was created to address the contemporary challenges and issues facing libraries. The system is bug-free and has the ability to add and validate users.

- **Reduce expenses**

  less human labor will be needed to maintain the library after the computerized system is put in place, which will lower total costs.

- **Conserve time**.

  The librarian can save precious time by searching records with a few mouse clicks and search terms.

- **Online notice board option**

  the librarian will be able to give a thorough explanation of the workshops taking place at the institution and other surrounding organizations.

  - Automate the cataloging and indexing of books.
  - Provide efficient search and retrieval capabilities for users.
  - Simplify the tracking of borrowed books and overdue fines.
  - Enhance accuracy and reduce human errors in record-keeping.
  - Easy operations for operator of the system

1.4 Methodology

The methodology for the Library Management System project encompasses several key components designed to ensure a comprehensive and effective implementation of the database system. This includes data collection techniques, database modeling approaches, and hardware and software requirements.

1.4.1 Data Collection Technique

To understand the specific needs and challenges faced by library staff and patrons, a combination of qualitative and quantitative data collection techniques will be employed:

-**Surveys and Questionnaires**: Distributed to library users and staff to gather insights on current challenges, desired features, and user experiences with existing systems.

-Interviews: Conducted with library management and staff to obtain detailed feedback on operational processes and the specific requirements for a new system.

- Observational Studies: Observing the daily operations within the library to identify pain points in the current manual system and areas for improvement.

- Existing Data Review: Analyzing current records and processes to assess data accuracy and identify the volume and types of information that need to be digitized.

1.4.2 Database Modeling Approach

The database modeling approach involves several steps to create a robust and efficient database structure:

1. Entity Identification: Determine the key entities involved in library operations, such as Books, Patrons, Loans, and Staff.

2. **Attribute Definition**: Define the attributes for each entity, ensuring that all relevant data is captured (e.g., book title, author, publication year, patron contact details, loan dates).

3. Relationship Mapping: Establish the relationships between entities, such as the relationship between Patrons and Loans, which indicates which patrons have borrowed which books.

4. Normalization: Apply normalization techniques to ensure data integrity and minimize redundancy, resulting in a well-structured relational database.

1.4.3 Hardware and Software Requirements

To successfully implement the Library Management System, specific hardware and software requirements must be met:

Hardware Requirements

- **My personal computer**

Device name:LAPTOP-1TIQ6DAT

Processor AMD Ryzen 3 3250U with Radeon Graphics 2.60 GHz

Installed RAM 8.00 GB (5.92 GB usable)

- **Server**: A dedicated server or cloud-based solution to host the database and handle user requests.
- **Workstations**: Computers for library staff equipped with adequate processing power and memory to run database applications.
- **Network Infrastructure**: A reliable network setup to ensure smooth connectivity between the server and user workstations, including routers and switches.

Software Requirements

- **Database Management System (DBMS):** A DBMS such as MySQL, PostgreSQL, or Oracle will be required to create and manage the database.

- **User Interface Software**: Development tools to create a user-friendly interface for both library staff and patrons, potentially using web-based technologies for accessibility.

- **Operating System:** Compatible operating systems (e.g., Windows, Linux) for the server and client machines.

- **Backup Solutions**: Software for regular data backups to prevent data loss and ensure data recovery in case of system failure.

By following this structured methodology, the Library Management System aims to enhance the efficiency of book records management, ultimately improving service delivery and user satisfaction in library environments.

Conclusion for Chapter

This chapter sets the stage for understanding the need for an LMS and how it can revolutionize library management. By highlighting the problems and potential solutions, it underscores the importance of transitioning from manual systems to an automated, efficient approach.

Chapter 2

Database Designing

The design of a robust database is essential for the effective implementation of the LMS. This chapter covers database modeling, business rules, building blocks, and the creation of an Entity-Relationship (ER) diagram. It also discusses the normalization process and the relational model mapping.

2.1 Database Modeling

Overview of Database Modeling: Structuring Data for Success

Database modeling is the process of designing a structured plan for organizing, storing, and accessing data in a database system. It bridges the gap between raw

data and actionable insights, ensuring the system is efficient, scalable, and tailored to user needs. Whether for a small application or a large enterprise system, database modeling forms the backbone of a solid data infrastructure.

Core Components of Database Modeling

1. **Conceptual Model: The Big Picture**

- ➢ Provides a high-level overview of the data and its relationships without diving into technical specifics.
- ➢ Tools like Entity-Relationship (ER) diagrams are used to illustrate entities.

2. **Logical Model: The Framework**

- ✓ Translates the conceptual design into a detailed structure, emphasizing data organization and adherence to rules like normalization.
- ✓ Focuses on defining attributes (columns) and relationships (keys).

3. **Physical Model: The Implementation**
    - Converts the logical structure into a real-world database setup tailored to a specific database management system (DBMS).
    - Addresses practical considerations such as storage formats, indexing, and query optimization.

Benefits of Database Modeling

⇒ **Ensures Data Integrity**: Establishes valid relationships and maintains consistency across the system.

⇒ **Improves Performance**: Lays a foundation for faster data retrieval and optimized storage.

⇒ **Simplifies Maintenance**: Offers a clear, organized design that aids developers and administrators.

⇒ **Supports Scalability**: Prepares the database for future growth and evolving requirements.

Insights from Experts

In the book *Database System Concepts*, authors Silberschatz, Korth, and Sudarshan emphasize the critical role of database modeling in creating efficient and scalable systems. They explain how a well-designed model captures the essence of business requirements while ensuring the database is robust and adaptable.

"An effective database model not only reflects current needs but also prepares for future challenges, providing a reliable and scalable foundation."
(Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts. McGraw-Hill, 6th Edition, 2010)

Database modeling is more than a technical task—it's a strategic approach to unlocking the potential of data. Whether designing a global application or a small-scale system, a thoughtfully crafted model is the cornerstone of success.

2.2 Business Rules and Requirements for the Library Management System

The success of the Library Management System hinges on clear and practical business rules that define how the system operates. These rules ensure smooth library operations, enhance user experience, and maintain data integrity. Here's an engaging breakdown of the key requirements for this project:

1. Unique Identification for Every Book

- **Rule**: Each book in the library must have a distinctive identifier, specifically an ISBN (International Standard Book Number).
- **Why It Matters**: This ensures that each book, regardless of title or edition, is uniquely identifiable, eliminating confusion and preventing duplication in the system.

2. User-Friendly Search Functionality

- **Rule**: Users should be able to search for books using key attributes such as title, author, and subject.
- **Why It Matters**: A robust search feature makes finding the right book quick and intuitive, enhancing user satisfaction and encouraging library use.

3. Comprehensive Borrowing Records

- **Rule**: Every borrowing transaction must capture the user's details, the book borrowed, and the due date.
- **Why It Matters**: Maintaining detailed records ensures accountability, tracks borrowing trends, and provides the data needed for issuing reminders or managing overdue books.

4. Overdue Fines Calculation

- **Rule**: The system must automatically calculate fines for books that are returned after their due date.
- **Why It Matters**: This promotes timely returns, ensures fair treatment of all users, and provides a steady mechanism to uphold library policies.

⇒ **Why These Rules Are Essential**

These business rules aren't just guidelines—they're the foundation of a well-organized and efficient Library Management System. They cater to the library's operational needs, create a seamless experience for users, and establish a fair and transparent system for borrowing and managing resources.

By adhering to these principles, the Library Management System will foster a user-centric environment while keeping the library's operations running smoothly.

2.3 Building Block of the Database

The **Building Blocks of a Database** refer to the fundamental elements that make up a database system. These elements are essential for designing, organizing, and managing data efficiently in a database. Each building block serves a specific role in structuring data and ensuring its integrity and accessibility. Here's an explanation of each term related to the fundamental building blocks of a database:

a. **Entities**

An **entity** is any object, thing, or concept in the real world that has data stored about it in the database. They are typically things that have attributes, and they can have relationships with other entities. In context of Library Management System, typical entities include:

1. **Patron**: Refers to the individuals who borrow books from the library.

2**. Book**: Represents the books available for borrowing.

3. **Loan**: Tracks the books that are borrowed, along with information on due dates, return dates, and loan status.

4. **Staff**: Represents the staff members who manage the library.

5. **Category**: Refers to the different classifications or genres of books in the library (e.g., fiction, non-fiction, mystery,etc.)

b. **Attributes**

Attributes are the properties or characteristics that describe an entity. For example, a "Book" entity might have attributes like "Title," "Author," "ISBN," and "Publication Year." Attributes give more specific details about entities and are represented as columns in a table.

c. **Relationship**

A **relationship** describes how two or more entities are related to each other. For example, a "Stuff" entity might be related to a "Loan" entity through a "managed" relationship, which signifies that students can enroll in courses. Relationships can be one-to-one, one-to-many, or many-to-many.

d. Primary Key

A **primary key** is an attribute (or a set of attributes) that uniquely identifies each record in a table. It must contain unique values and cannot contain NULLs. For example

Patron: PatronID (Unique identifier for each patron)

- Book: BookID (Unique identifier for each book)

- Loan: LoanID (Unique identifier for each loan)

-Staff: StaffID (Unique identifier for each staff member)

- Category: CategoryID (Unique identifier for each category)

Primary keys guarantee that each entry in the database is identifiable, eliminating duplicates and ensuring data integrity.

e. Foreign Key

A **foreign key** is an attribute in one table that links to the primary key of another table. It establishes a relationship between two tables. For example, In the Loan entity, PatronID serves as a foreign key linking the loan to the patron who borrowed the book.

f. Normalization

Normalization is the process of organizing the data in a database to reduce redundancy and improve data integrity. The goal of normalization is to ensure that each piece of data is stored only once and that the data dependencies are logical. The process involves breaking down large tables into smaller, related ones and ensuring that each table represents a single concept.

g. ER Diagram

An Entity-Relationship (ER) Diagram is a visual representation of the database structure. It shows the entities, their attributes, and the relationships between them. For a Library Management System, An Entity-Relationship (ER) Diagram for a "Book" might look like this:

- **Entities**: Book, Author, Publisher
- **Attributes of Book**: ISBN (Primary Key), Title, Publication Year
- **Attributes of Author**: Author_ID (Primary Key), Name, Date of Birth
- **Attributes of Publisher**: Publisher_ID (Primary Key), Name, Address

2.4 ER Diagram

An Entity-Relationship (ER) diagram visually represents the relationships between entities in the database. It is a crucial step in database design, ensuring all necessary data and relationships are captured.

To create an effective Entity-Relationship (ER) diagram for a Library Management System (LMS), we can identify the main entities, attributes, and relationships based on the considerations and steps discussed in your document. Here's a breakdown of the key components that would typically be included in such an ER diagram:

Entities and Attributes

1. **Patron**

PatronID	Name	Address	Phone Number	Email	Membership Date

2. **Book**

BookID	Title	Author	ISBN	Publisher	Publication Year	Genre	Copies Available

3. **Loan**

LoanID	LoanDate	DueDate	ReturnDate	Status (e.g., active, completed)

4. **Staff**

StaffID	Name	Position	Email	Phone Number

5. **Category**

CategoryID	Category Name

2.5 Relationships

1. **Patron to Loan**

- A Patron can have multiple Loans (1-to-Many relationship)

- A Loan is associated with one Patron

2. **Book to Loan**

- A Book can be loaned out multiple times (1-to-Many relationship)

- A Loan is associated with one Book

3. Book to Category

- A Book can belong to one or more Categories (Many-to-Many relationship)

- A Category can include multiple Books

4. Staff to Book

- A Staff member can manage multiple Books (1-to-Many relationship)

- A Book is managed by one Staff member

Diagram Overview

- Patron connects to Loan with a line indicating a 1-to-Many relationship.

- Book connects to Loan with a line indicating a 1-to-Many relationship.

- Book connects to Category with a dashed line indicating a Many-to-Many relationship (often requiring a junction table).

- Staff connects to Book with a line indicating a 1-to-Many relationship.

This conceptual framework serves as the basis for creating the ER diagram. The relationships between these entities should be clearly represented with arrows

indicating the nature and cardinality of each relationship, while attributes should be listed under their respective entities.

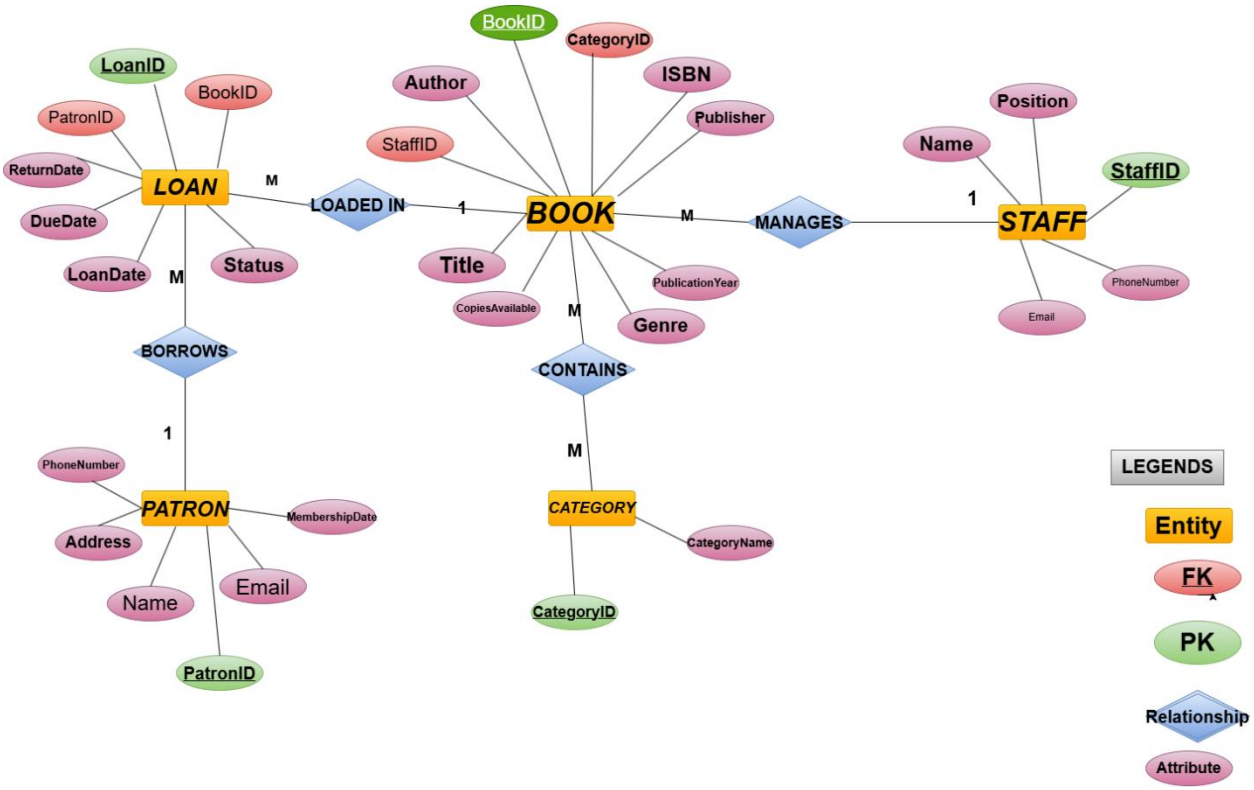


Figure 2.1 ER Diagram

2.5 Relational Model: Mapping ER Diagram to Relational diagram Model

The Relational Model is a foundational concept in database management systems that organizes data into structured tables known as relations. Each table

consists of rows (tuples) and columns (attributes). The key characteristics of the Relational Model include:

1. Data Structure: Data is organized in tables with a predefined schema. Each table is defined by its attributes, each of which has a specified data type.

2. Relationships: Relationships between tables (relations) are established using foreign keys, which are attributes in one table that refer to the primary key in another table, thus ensuring data integrity and enabling various operations through joins.

3. Normalization: A crucial aspect of the Relational Model is normalization, which is the process of structuring tables to minimize redundancy and dependency, ensuring efficient data management and integrity.

4. Operations: The Relational Model includes a set of operations like select, project, join, and union, which can be applied to manipulate and retrieve data efficiently.

Mapping an Entity-Relationship (ER) diagram to a Relational Model involves transforming the conceptual design (ER diagram) into a logical design (relational schema). This process can be broken down into several steps:

Step 1: Identify Entities and Attributes

Entities in the ER diagram correspond to tables in the Relational Model. Each entity is associated with attributes that become the columns of its respective table. For instance, if we have an entity called "Staff" with attributes such as StaffID, Name, and Email, this will result in a table named "Staff" with columns for each attribute.

Step 2: Define Primary Keys

Every table must have a primary key, which uniquely identifies each record in the table. In our "Staff" example, StaffID can be the primary key because it uniquely identifies each customer. If an entity does not have a natural primary key, a surrogate key can be introduced.

Step 3: Identify Relationships and Foreign Keys

In the ER diagram, relationships among entities must be identified. These relationships determine how tables will be associated. For example, if there is a relationship between "Staff" and "Book" where a customer can place many orders, we need to establish a foreign key in the "Book" table.

- **One-to-One Relationships**: If two entities have a one-to-one relationship, the primary key from one entity can be included as a foreign key in the other entity's table.

-**One-to-Many Relationships**: In a one-to-many relationship, the primary key of the "One" side becomes a foreign key in the "Many" side. For instance, if a Staff can place multiple Books, then StaffID (primary key from the "Staff" table) is a foreign key in the "Book" table.

- **Many-to-Many Relationships**: For many-to-many relationships, a junction table (also known as a bridge table) is created that contains foreign keys referencing the primary keys of the two related entities.

Step 4: Map Attributes and Include Constraints

Each attribute from the ER entities translates directly into table columns. Data types should be specified based on the nature of the data. Additionally, constraints like NOT NULL, UNIQUE, and CHECK can be defined to enforce business rules and maintain data integrity. For example, an email attribute in the "Customer" table can be made unique to prevent duplicate email entries.

Step 5: Review the Relational Schema

Once the mappings and constraints are established, a comprehensive review of the relational schema is essential. This includes verifying that the schema is normalized to minimize redundancy and ensure data integrity and performance.

The relational model transforms the ER diagram into tables with defined relationships, ensuring data integrity and facilitating efficient data retrieval.

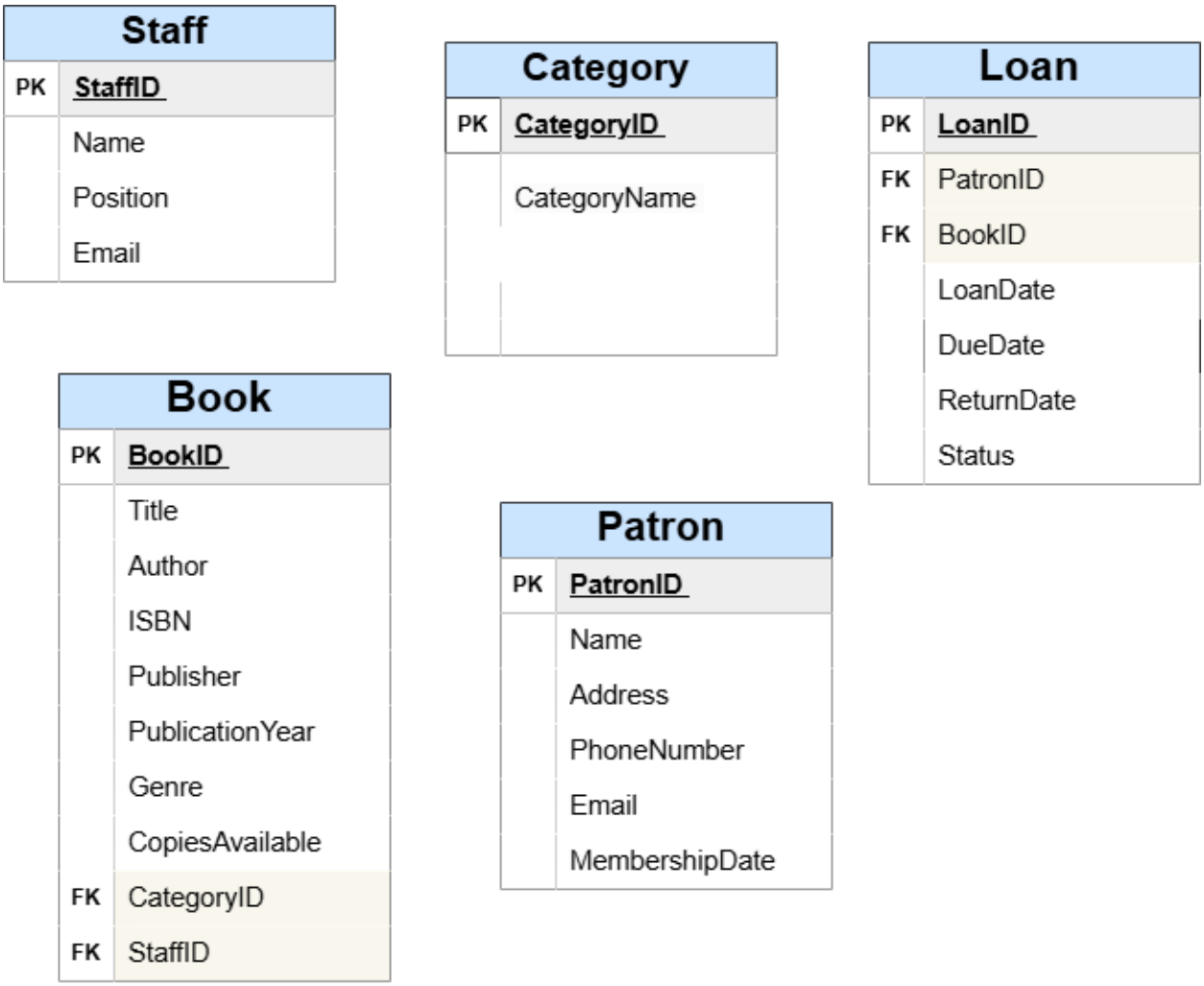


Figure 2.2 Relational Model

2.6 Normalization

Introduction to Normalization in a Library Management System

Normalization is a fundamental process in database design that aims to organize data to reduce redundancy and improve data integrity. In the context of a library management system, normalization ensures that the database is structured efficiently, making it easier to manage and retrieve information about books, patrons, loans, staff, and categories. This introduction will cover the basics of

normalization, its importance, and how it can be applied to a library management system.

What is Normalization?

Normalization is the process of organizing data in a database into tables and columns to minimize redundancy and dependency. The primary goal is to divide large tables into smaller, related tables and link them using relationships. This process involves applying a series of rules called normal forms, each of which addresses specific types of redundancy and anomalies.

The most commonly used normal forms are:

1. **First Normal Form (1NF)**: Ensures that the table has a primary key and that all columns contain atomic (indivisible) values.
2. **Second Normal Form (2NF)**: Builds on 1NF by ensuring that all non-key attributes are fully dependent on the primary key.
3. **Third Normal Form (3NF)**: Builds on 2NF by ensuring that all non-key attributes are not only fully dependent on the primary key but also independent of each other.
4. **Boyce-Codd Normal Form (BCNF):** is an advanced version of the Third Normal Form (3NF) and is used to ensure that the database design is free from redundancy and anomalies. BCNF is a stricter form of normalization that addresses certain types of anomalies that 3NF does not cover. In the context of a library management system, BCNF helps in organizing the data efficiently, ensuring data integrity, and eliminating redundancy.

Importance of Normalization

Normalization offers several benefits, including:

1. **Data Integrity**: By eliminating redundancy, normalization ensures that data is consistent and accurate. For example, in a library management system, information about a book's author should be stored in one place, reducing the risk of discrepancies.
2. **Efficient Data Retrieval**: Normalized databases are structured in a way that makes querying more efficient. This is crucial for a library management system where quick access to information about books, patrons, and loans is essential.
3. **Reduced Data Redundancy**: Normalization minimizes duplicate data, which saves storage space and reduces the likelihood of data anomalies. For instance, storing a patron's contact information in one table prevents the need to duplicate this information in multiple places.
4. **Simplified Maintenance**: With a normalized database, updates and deletions are easier to manage. Changes to data, such as updating a book's details or a patron's address, can be made in one place without affecting other parts of the database.

Normalization of Library Database Management System Entities

A) Normalization of the Relation/table Patron

Normalization of the Patron Entity in a Library Management System

The original patron table includes:

- PatronID(PRIMARY KEY)
- Name
- Address
- PhoneNumber
- Email
- Membership

First Normal Form (1NF)

A table is in First Normal Form (1NF) if it only contains atomic (indivisible) values and each column contains values of a single type. There should be no repeating groups or arrays.

Application to Patron Entity:

- Ensure all columns contain atomic values.
- Remove any duplicate columns or repeating groups.



Second Normal Form (2NF): A table is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. There should be no partial dependency, which means no attribute should depend on only a part of the primary key.

Application to Patron Entity:

- The Patron table is already in 2NF because all non-key attributes (Name, Address, PhoneNumber, Email, MembershipDate) are fully functionally dependent on the primary key (PatronID).

Third Normal Form (3NF): A table is in Third Normal Form (3NF) if it is in 2NF and all the attributes are functionally dependent only on the primary key. There should be no transitive dependency, meaning no non-key attribute should depend on another non-key attribute.

Application to Patron Entity:

- The Patron table is already in 3NF because there are no transitive dependencies. All non-key attributes are directly dependent on the primary key (PatronID).

Boyce-Codd Normal Form (BCNF):A table is in Boyce-Codd Normal Form (BCNF) if it is in 3NF, and for every functional dependency X→YX \rightarrow Y, XX must be a super key. BCNF is a stricter form of 3NF.

Patron Table in BCNF:

- The Patron table is already in BCNF because the only dependencies are on the primary key (PatronID), which is a super key.

B) Normalization of the Relation/table for Loan

The normalization process for the Loan entity in a library management system, ensuring it adheres to the principles of 1NF, 2NF, 3NF, and BCNF.

Original Loan Table:

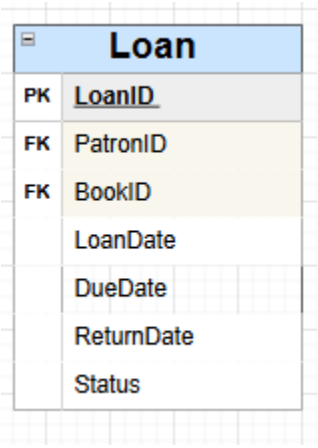
Assume the original Loan table has the following attributes:

- LoanID (Primary Key)

- PatronID (Foreign Key)
- BookID (Foreign Key)
- LoanDate
- DueDate
- ReturnDate
- Status

First Normal Form (1NF): A table is in First Normal Form (1NF) if it only contains atomic (indivisible) values, and each column contains values of a single type. There should be no repeating groups or arrays.

Loan Table in 1NF:



- **LoanID**: Unique identifier for each loan (Primary Key).
- **PatronID**: The ID of the patron who borrowed the book.
- **BookID**: The ID of the book that was borrowed.
- **LoanDate**: The date the book was borrowed.
- **DueDate**: The date the book is due to be returned.
- **ReturnDate**: The date the book was returned.
- **Status**: The status of the loan (e.g., active, completed).

Second Normal Form (2NF): A table is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. There should be no partial dependency, which means no attribute should depend on only a part of the primary key.

Loan Table in 2NF:

- Since the Loan table already has a single primary key (LoanID) and all other attributes are fully functionally dependent on LoanID, it is already in 2NF.

Third Normal Form (3NF): A table is in Third Normal Form (3NF) if it is in 2NF and all the attributes are functionally dependent only on the primary key. There should be no transitive dependency, meaning no non-key attribute should depend on another non-key attribute.

Loan Table in 3NF: The table already satisfies 3NF as there are no transitive dependencies.

Boyce-Codd Normal Form (BCNF): A table is in Boyce-Codd Normal Form (BCNF) if it is in 3NF, and for every functional dependency X→YX \right arrow Y, X X must be a super key.

Loan Table in BCNF: The table already satisfies BCNF as the primary key (LoanID) is a super key

C. Normalization of the Relation/table for Book

Entity: Book

The Original Attributes:

- **BookID**: Unique identifier for each book (Primary Key).
- **Title**: The title of the book.
- **Autho**r: The author of the book.
- **ISBN**: The International Standard Book Number, a unique identifier for books.
- **Publisher**: The publisher of the book.

- **PublicationYear**: The year the book was published.
- **Genre**: The genre of the book.
- **CopiesAvailable**: The number of copies available in the library.
- **CategoryID**: Foreign key referencing the Category entity.
- **StaffID**: Foreign key referencing the Staff entity.

First Normal Form (1NF): A table is in First Normal Form (1NF) if it contains only atomic (indivisible) values, and each column contains values of a single type. There should be no repeating groups or arrays.

Book Table in 1NF: The table already satisfies 1NF as all columns contain atomic values.



Second Normal Form (2NF): A table is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. There should be no partial dependency, which means no attribute should depend on only a part of the primary key.

Book Table in 2NF: The table already satisfies 2NF as all non-key attributes are fully functionally dependent on the primary key (BookID).

Third Normal Form (3NF): A table is in Third Normal Form (3NF) if it is in 2NF and all the attributes are functionally dependent only on the primary key. There should be no transitive dependency, meaning no non-key attribute should depend on another non-key attribute.

Book Table in 3NF: The table already satisfies 3NF as there are no transitive dependencies.

Boyce-Codd Normal Form (BC): A table is in Boyce-Codd Normal Form (BCNF) if it is in 3NF, and for every functional dependency X→YX \rightarrow Y, XX must be a super key.

Book Table in BCNF: The table already satisfies BCNF as the primary key (BookID) is a super key.

C) Normalization of the Relation/table for Staff

Entity: Staff

Attributes:

- **StaffID**: Unique identifier for each staff member (Primary Key).
- **Name**: The name of the staff member.
- **Position**: The job position or title of the staff member.
- **Email**: The email address of the staff member.
- **PhoneNumber**: The phone number of the staff member.

First Normal Form (1NF): A table is in First Normal Form (1NF) if it contains only atomic (indivisible) values, and each column contains values of a single type. There should be no repeating groups or arrays.

Staff Table in 1NF:



Second Normal Form (2NF): A table is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. There should be no partial dependency, which means no attribute should depend on only a part of the primary key.

Staff Table in 2NF: The table already satisfies 2NF as all non-key attributes are fully functionally dependent on the primary key (StaffID).

Third Normal Form (3NF): A table is in Third Normal Form (3NF) if it is in 2NF and all the attributes are functionally dependent only on the primary key. There should be no transitive dependency, meaning no non-key attribute should depend on another non-key attribute.

Staff Table in 3NF: The table already satisfies 3NF as there are no transitive dependencies.

Boyce-Codd Normal Form (BCNF): A table is in Boyce-Codd Normal Form (BCNF) if it is in 3NF, and for every functional dependency X→YX \rightarrow Y, XX must be a super key.

Staff Table in BCNF: The table already satisfies BCNF as the primary key (StaffID) is a super key

E. Normalization of the Relation/table for Category

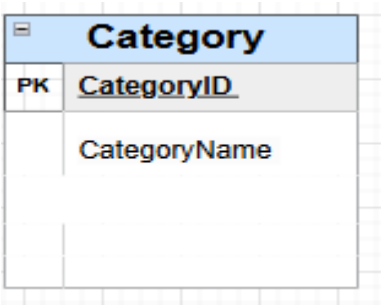
Entity: Category

Attributes:

- **CategoryID**: Unique identifier for each category (Primary Key).
- **CategoryName**: The name of the category.

First Normal Form (1NF): A table is in First Normal Form (1NF) if it contains only atomic (indivisible) values, and each column contains values of a single type. There should be no repeating groups or arrays.

Category Table in 1NF:



Second Normal Form (2NF): A table is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. There should be no partial dependency, which means no attribute should depend on only a part of the primary key.

Category Table in 2NF: The table already satisfies 2NF as all non-key attributes are fully functionally dependent on the primary key (CategoryID).

Third Normal Form (3NF): A table is in Third Normal Form (3NF) if it is in 2NF and all the attributes are functionally dependent only on the primary key. There should be no transitive dependency, meaning no non-key attribute should depend on another non-key attribute.

Category Table in 3NF: The table already satisfies 3NF as there are no transitive dependencies.

Boyce-Codd Normal Form (BCNF): A table is in Boyce-Codd Normal Form (BCNF) if it is in 3NF, and for every functional dependency X→YX \rightarrow Y, XX must be a super key.

Category Table in BCNF: The table already satisfies BCNF as the primary key (CategoryID) is a super key.

Conclusion for Chapter

A well-structured database model is critical for achieving operational efficiency and data integrity in a Library Management System. The design presented ensures scalability and meets the specific requirements of modern libraries, forming a solid foundation for implementation.

Chapter 3

Database Implementation

This chapter focuses on the practical aspects of implementing the database designed for the LMS. It includes schema definitions, SQL commands, and a relational diagram. The emphasis is on ensuring that the database supports all operational needs effectively and efficiently.

The introduction of database Implementation for library management system

Designing and implementing a database for a library management system involves creating a structured framework to efficiently manage library operations, including tracking books, patrons, borrowing activities, and administrative tasks. A well-designed database ensures data integrity, supports efficient queries, and facilitates seamless library management.

The implementation phase involves creating the database using a Database Management System (DBMS) such as MySQL or PostgreSQL. It includes:

1. **Schema Definition**: Defining tables, fields, and data types.

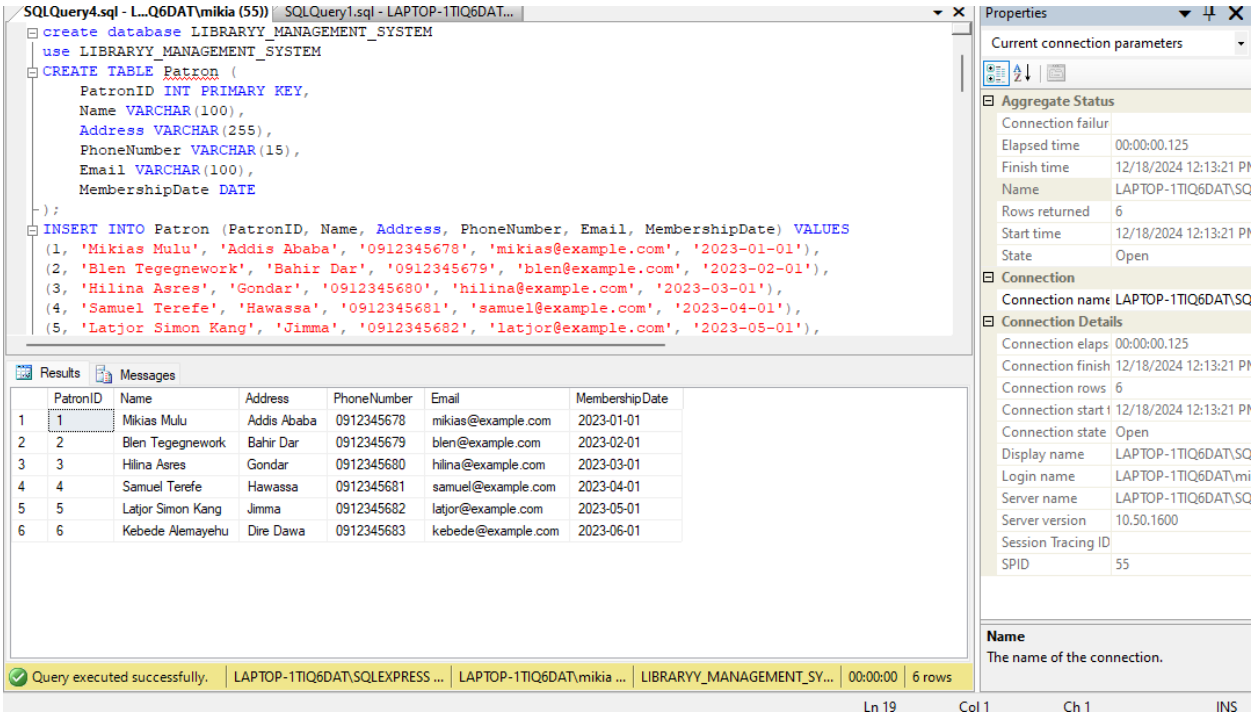
2. **Constraints**: Setting primary and foreign keys, and other constraints for data integrity.

3. **Data Population**: Loading initial data into the database.

4. **Query Optimization**: Ensuring efficient data retrieval and manipulation through optimized queries.

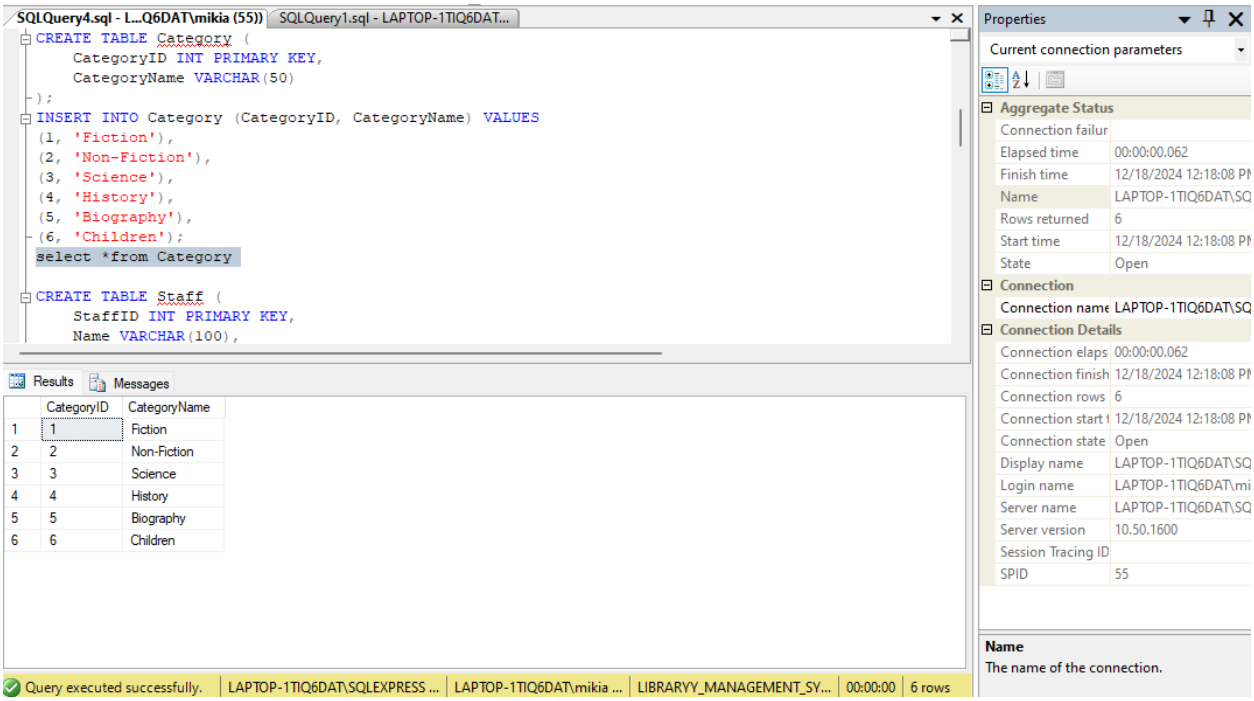
By implementing a well-designed database system, libraries can overcome the challenges of managing and accessing book records, leading to improved operational efficiency and user satisfaction. The system not only automates routine tasks but also provides valuable insights through data analytics, ultimately enhancing the library's service offerings.

3.2 Whole SQL Command to develop LMS database with some inserted values

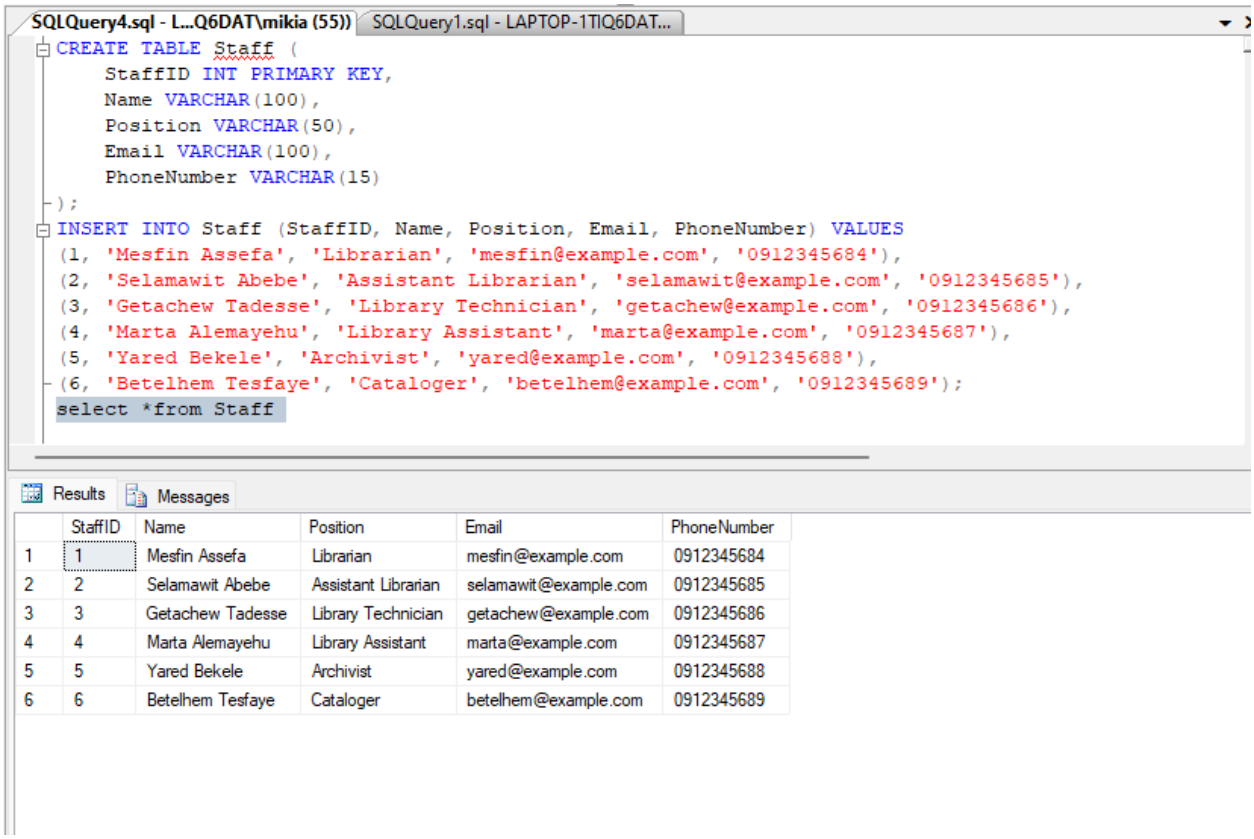



```
create database LIBRARYY_MANAGEMENT_SYSTEM
use LIBRARYY_MANAGEMENT_SYSTEM
CREATE TABLE Patron (
    PatronID INT PRIMARY KEY,
    Name VARCHAR(100),
    Address VARCHAR(255),
    PhoneNumber VARCHAR(15),
    Email VARCHAR(100),
    MembershipDate DATE
);
INSERT INTO Patron (PatronID, Name, Address, PhoneNumber, Email,
MembershipDate) VALUES
(1, 'Mikias Mulu', 'Addis Ababa', '0912345678', 'mikias@example.com', '2023-
01-01'),
(2, 'Blen Tegegnework', 'Bahir Dar', '0912345679', 'blen@example.com', '2023-
02-01'),
```

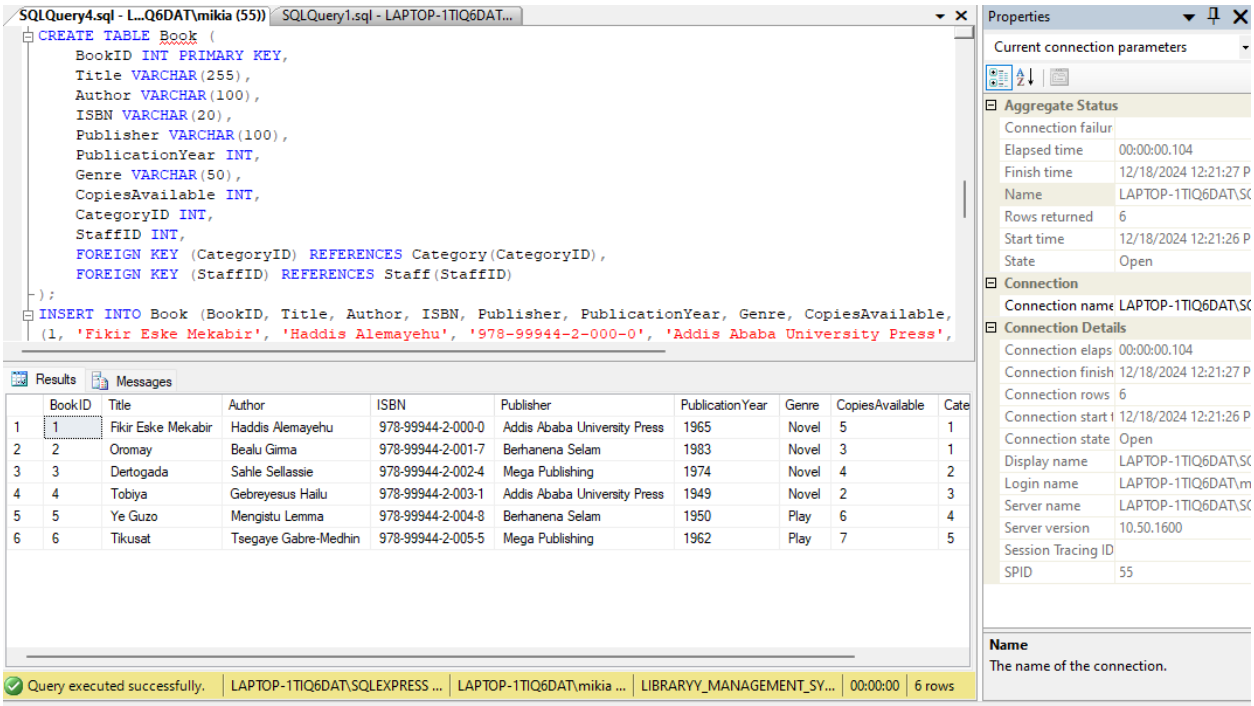
```
(3, 'Hilina Asres', 'Gondar', '0912345680', 'hilina@example.com', '2023-03-
01'),
(4, 'Samuel Terefe', 'Hawassa', '0912345681', 'samuel@example.com', '2023-04-
01'),
(5, 'Latjor Simon Kang', 'Jimma', '0912345682', 'latjor@example.com', '2023-
05-01'),
(6, 'Kebede Alemayehu', 'Dire Dawa', '0912345683', 'kebede@example.com',
'2023-06-01');
select *from Patron
```



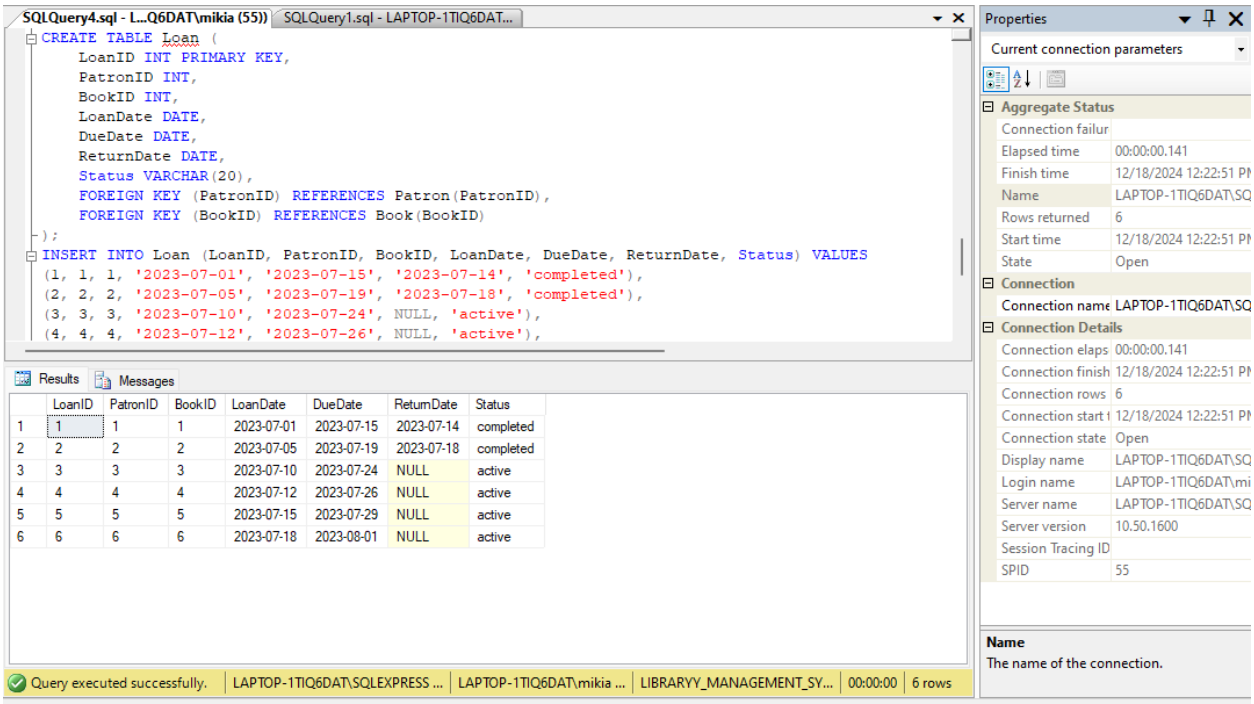

```
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50)
);
INSERT INTO Category (CategoryID, CategoryName) VALUES
(1, 'Fiction'),
(2, 'Non-Fiction'),
(3, 'Science'),
(4, 'History'),
(5, 'Biography'),
(6, 'Children');
select *from Category
```




```
CREATE TABLE Staff (
    StaffID INT PRIMARY KEY,
    Name VARCHAR(100),
    Position VARCHAR(50),
    Email VARCHAR(100),
    PhoneNumber VARCHAR(15)
);
INSERT INTO Staff (StaffID, Name, Position, Email, PhoneNumber) VALUES
(1, 'Mesfin Assefa', 'Librarian', 'mesfin@example.com', '0912345684'),
(2, 'Selamawit Abebe', 'Assistant Librarian', 'selamawit@example.com',
'0912345685'),
(3, 'Getachew Tadesse', 'Library Technician', 'getachew@example.com',
'0912345686'),
(4, 'Marta Alemayehu', 'Library Assistant', 'marta@example.com',
'0912345687'),
(5, 'Yared Bekele', 'Archivist', 'yared@example.com', '0912345688'),
(6, 'Betelhem Tesfaye', 'Cataloger', 'betelhem@example.com', '0912345689');
select *from Staff
```




```
CREATE TABLE Book (
    BookID INT PRIMARY KEY,
    Title VARCHAR(255),
    Author VARCHAR(100),
    ISBN VARCHAR(20),
    Publisher VARCHAR(100),
    PublicationYear INT,
    Genre VARCHAR(50),
    CopiesAvailable INT,
    CategoryID INT,
    StaffID INT,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID)
);
INSERT INTO Book (BookID, Title, Author, ISBN, Publisher, PublicationYear,
Genre, CopiesAvailable, CategoryID, StaffID) VALUES
(1, 'Fikir Eske Mekabir', 'Haddis Alemayehu', '978-99944-2-000-0', 'Addis
Ababa University Press', 1965, 'Novel', 5, 1, 1),
(2, 'Oromay', 'Bealu Girma', '978-99944-2-001-7', 'Berhanena Selam', 1983,
'Novel', 3, 1, 2),
(3, 'Dertogada', 'Sahle Sellassie', '978-99944-2-002-4', 'Mega Publishing',
1974, 'Novel', 4, 2, 3),
(4, 'Tobiya', 'Gebreyesus Hailu', '978-99944-2-003-1', 'Addis Ababa
University Press', 1949, 'Novel', 2, 3, 4),
(5, 'Ye Guzo', 'Mengistu Lemma', '978-99944-2-004-8', 'Berhanena Selam',
1950, 'Play', 6, 4, 5),
(6, 'Tikusat', 'Tsegaye Gabre-Medhin', '978-99944-2-005-5', 'Mega
Publishing', 1962, 'Play', 7, 5, 6);
select *from Book
```




```
CREATE TABLE Loan (
    LoanID INT PRIMARY KEY,
    PatronID INT,
    BookID INT,
    LoanDate DATE,
    DueDate DATE,
    ReturnDate DATE,
    Status VARCHAR(20),
    FOREIGN KEY (PatronID) REFERENCES Patron(PatronID),
    FOREIGN KEY (BookID) REFERENCES Book(BookID)
);
INSERT INTO Loan (LoanID, PatronID, BookID, LoanDate, DueDate, ReturnDate,
Status) VALUES
(1, 1, 1, '2023-07-01', '2023-07-15', '2023-07-14', 'completed'),
(2, 2, 2, '2023-07-05', '2023-07-19', '2023-07-18', 'completed'),
(3, 3, 3, '2023-07-10', '2023-07-24', NULL, 'active'),
(4, 4, 4, '2023-07-12', '2023-07-26', NULL, 'active'),
(5, 5, 5, '2023-07-15', '2023-07-29', NULL, 'active'),
(6, 6, 6, '2023-07-18', '2023-08-01', NULL, 'active');
select *from Loan
```

3.3 Relational diagram

The **relational model** is a database model that organizes data into **tables (also known as relations)**, where each table consists of **rows (records)** and **columns**

(attributes). This model is based on mathematical principles of set theory and predicate logic, ensuring a systematic and logical representation of data.

In the relational model:

Tables represent entities or objects (e.g., books, members).

Rows represent individual instances of those entities (e.g., a specific book or member).

Columns define the attributes of those entities (e.g., title, author, ISBN for books).

Relationships between entities are established using **primary keys** (unique identifiers in a table) and **foreign keys** (columns referencing primary keys in another table).

Why Is the Relational Model Important for a Library Management System?

The relational model provides several advantages that make it particularly suitable for a library management system:

1. Data Organization and Integrity

The relational model allows for clear organization of library data into separate tables such as:

- Books (ISBN, title, author, genre, availability status)
- Members (member ID, name, contact information, membership type)
- Transactions (transaction ID, book ID, member ID, issue date, return date)

Data integrity is maintained through constraints, ensuring consistent and accurate information across the system.

2. Support for Relationships

A library system requires relationships between entities. For instance:

A **member** can borrow multiple **books**, but each borrowing transaction is specific to one book and one member.

- These relationships are easily represented through **foreign keys**.

Example:

Transactions table has a member_id column referencing the Members table and a book_id column referencing the Books table.

3. Scalability and Flexibility

New data or entities (e.g., e-books, inter-library loans) can be added without disrupting existing structures.

- Queries can adapt to evolving requirements, such as retrieving overdue books or most-borrowed titles.

4. Efficient Querying

Using SQL (Structured Query Language), a relational database enables efficient data retrieval, such as:

- ✓ Checking availability of a specific book.
- ✓ Listing all books borrowed by a member.
- ✓ Tracking overdue items.

5. **Data Consistency and Redundancy Reduction**

By normalizing the database, redundant data is minimized. For example:

- Instead of repeating author details in every book record, a separate Authors table can store author information linked to the Books table.
- This ensures consistency and reduces storage requirements.

6. **Enforcing Business Rules**

Constraints like:

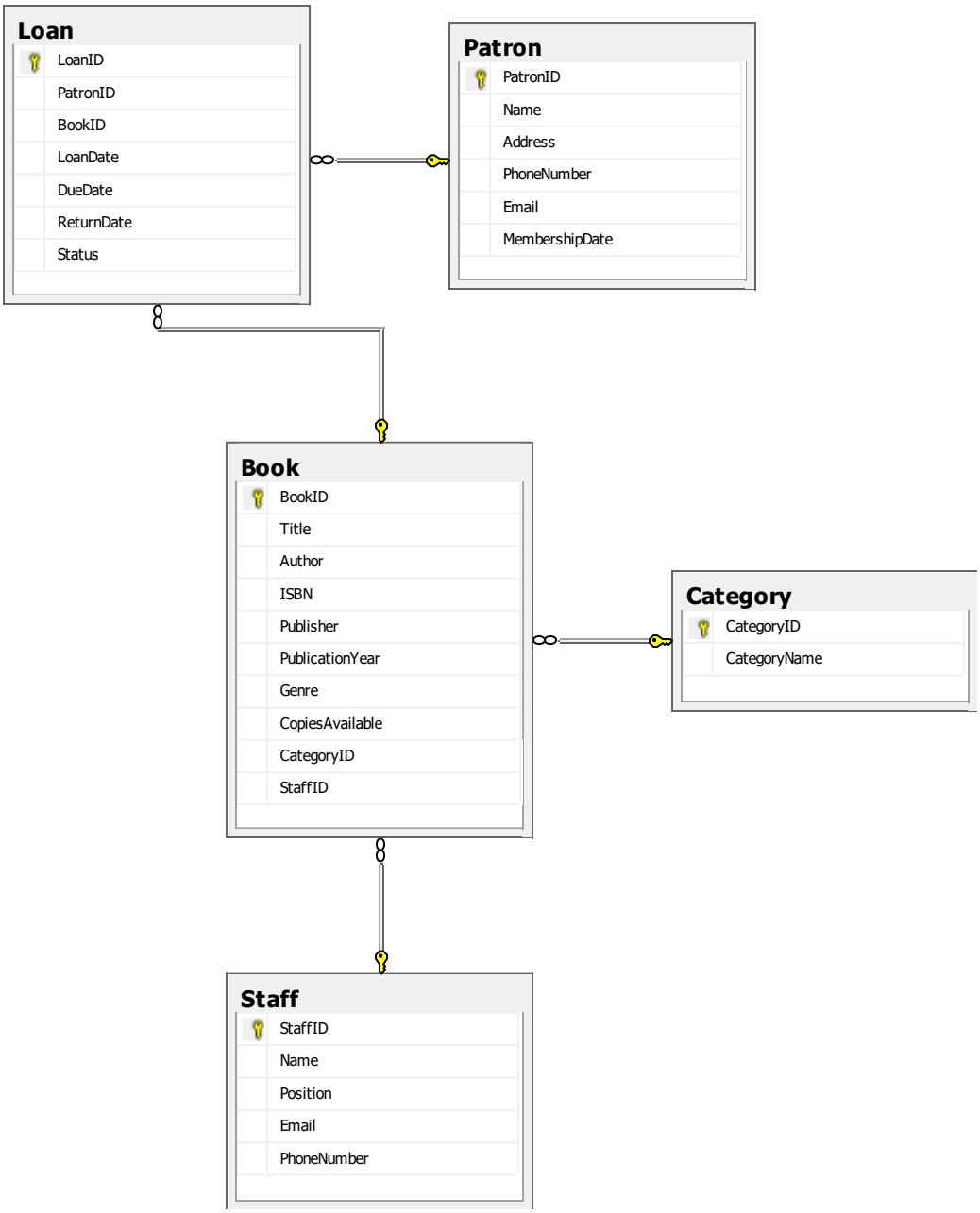
- A book cannot be issued if it's already borrowed.
- Membership must be valid to issue books.
- These rules can be implemented using relational database constraints.

7. Transaction Management

Supports features like atomicity, consistency, isolation, and durability (ACID properties), ensuring reliable data handling during simultaneous user interactions.

Conclusion for Chapter

The implementation of the LMS database transforms the theoretical design into a functional system. By leveraging SQL and relational principles, the database supports seamless library operations, ensuring accurate data handling and improved user satisfaction.


Loan
LoanID
PatronID
BookID
LoanDate
DueDate
ReturnDate
Status
Patron
PatronID
Name
Address
PhoneNumber
Email
MembershipDate
Book
BookID
Title
Author
ISBN
Publisher
PublicationYear
Genre
CopiesAvailable
CategoryID
StaffID
Category
CategoryID
CategoryName
Staff
StaffID
Name
Position
Email
PhoneNumber

Reference

- "Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan.
- "Database Management Systems" by Raghu Ramakrishnan and Johannes Gehrke.
- **Chen, W.** (2023). *Database Design and Implementation*
- **Codd, E. F.** (1990). *The Relational Model for Database Management: Version 2*. Addison-Wesley.
- **Harrington, J. L.** (2016). *Relational Database Design and Implementation: Clearly Explained* (4th ed.). Morgan Kaufmann.